

UNIT-5

TRANSACTION MANAGEMENT AND INTRODUCTION TO INDEXING

What is a Transaction?

1. A transaction is an **event which occurs on the database.**
2. Database **Transaction is an atomic unit** that contains one or more SQL statements.
3. Generally a transaction **reads a value** from the database or **writes a value** to the database.
4. **It is a series of operations** that performs as a single unit of work against a database.

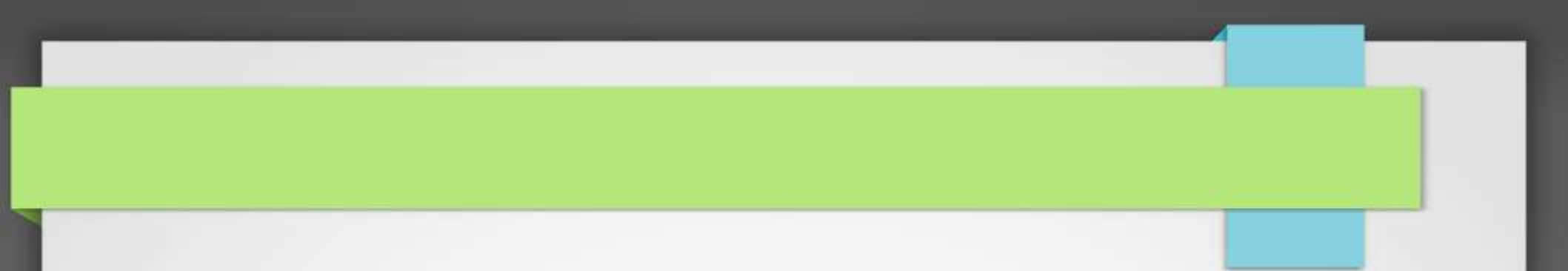
E.g., Transaction

E.g., transaction to **transfer \$50 from account A to account B:**

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Example

Before:- X-500	Y-400
Transaction T	
T1	T2
Read (X) X: = X-100 Write(X)	Read (Y) Y: = Y+100 Write(Y)
After:- X : 400	After:- Y : 400



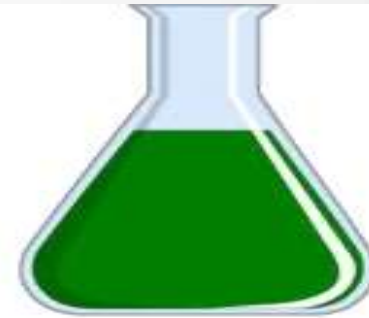
Transaction Properties (ACID)

Transaction Properties(ACID)

Following are the Transaction Properties, referred to by an acronym ACID properties:

ACID PROPERTIES

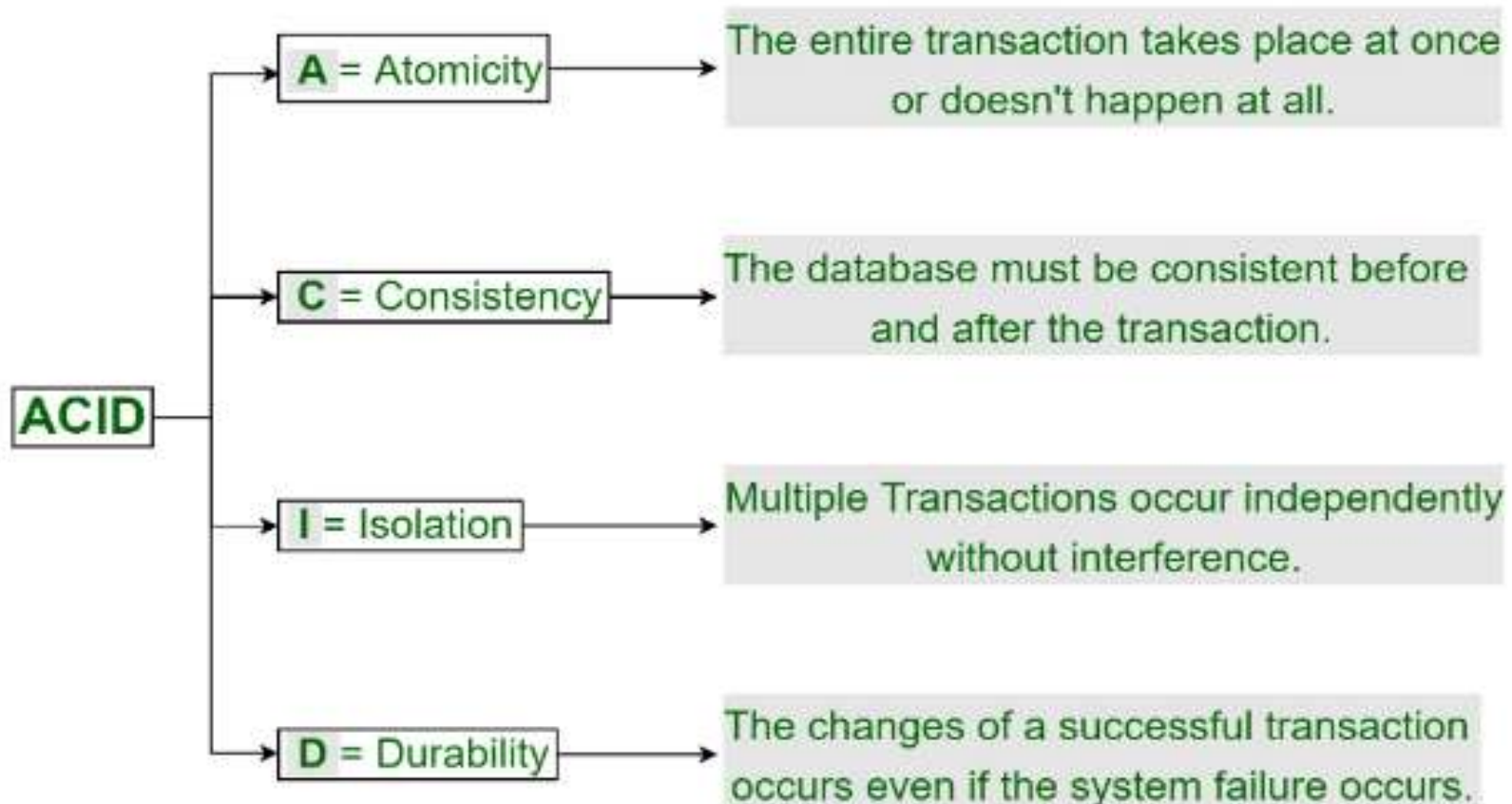
To preserve the integrity of data the database system must ensure ACID properties



- A** Atomicity
- C** Consistency
- I** Isolation
- D** Durability

Transaction Properties(ACID)

ACID Properties in DBMS



Transaction Properties(ACID)

Atomicity

1. Atomicity defines that **all operations** of the transactions are **either executed or none**.
2. Atomicity is also known as '**All or Nothing**', it means that either perform the operations or not perform at all.
3. It is maintained in the **presence of deadlocks, CPU failures**, disk failures, database and application software failures.
4. It can be turned off at **system level and session level**.

Atomicity

Before:- X-500	Y-400
Transaction T	
T1	T2
Read (X) X: = X-100 Write(X)	Read (Y) Y: = Y+100 Write(Y)
After:- X : 400	After:- Y : 400

For example,

We have two accounts A and B, each containing Rs 1000/-. We now start a transaction to deposit Rs 100/- from account A to Account B.

```
Read A;  
A = A - 100;  
Write A;  
Read B;  
B = B + 100;  
Write B;
```

Fine, is not it? **The transaction has 6 instructions** to extract the amount from A and submit it to B.

The AFIM will show Rs 900/- in A and Rs 1100/- in B.

For example,

Now, suppose there is a **power failure just after instruction 3** (Write A) has been complete.

What happens now?

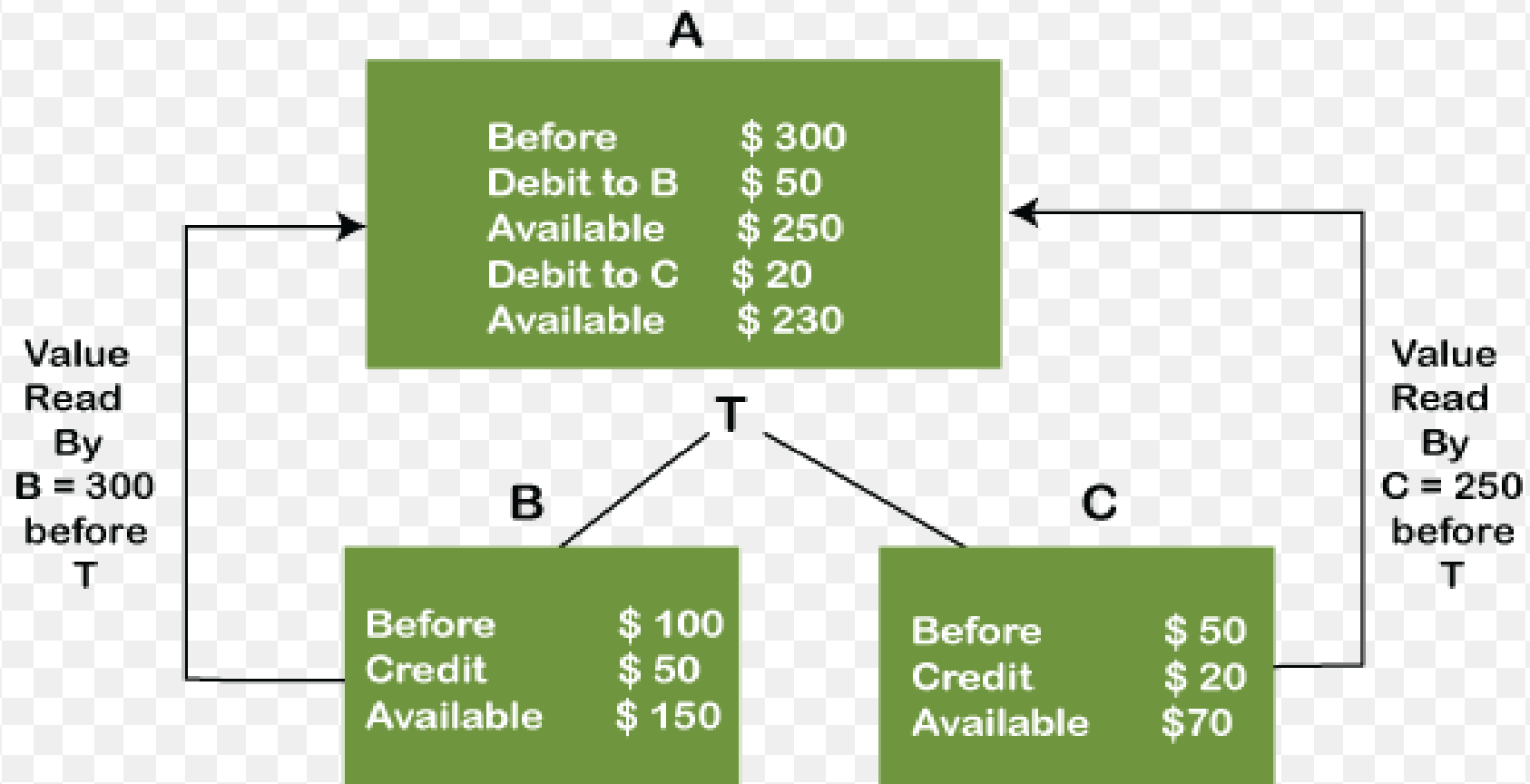
- ✓ After the system recovers the AFIM will show Rs 900/- in A, but the same Rs 1000/- in B.
- ✓ It would be said that Rs 100/- evaporated in thin air for the power failure.
- ✓ **Clearly such a situation is not acceptable.**

Transaction Properties(ACID)

Consistency

1. Consistency defines that after the **transaction is finished**, the **database must remain in a consistent state**.
2. It preserves consistency of the database.
3. **If the transaction fails**, then the transaction will be **rolled back** and the database will be restored to a state consistent.
4. **Consistency is said to be failed if atomicity fails.**

Consistency



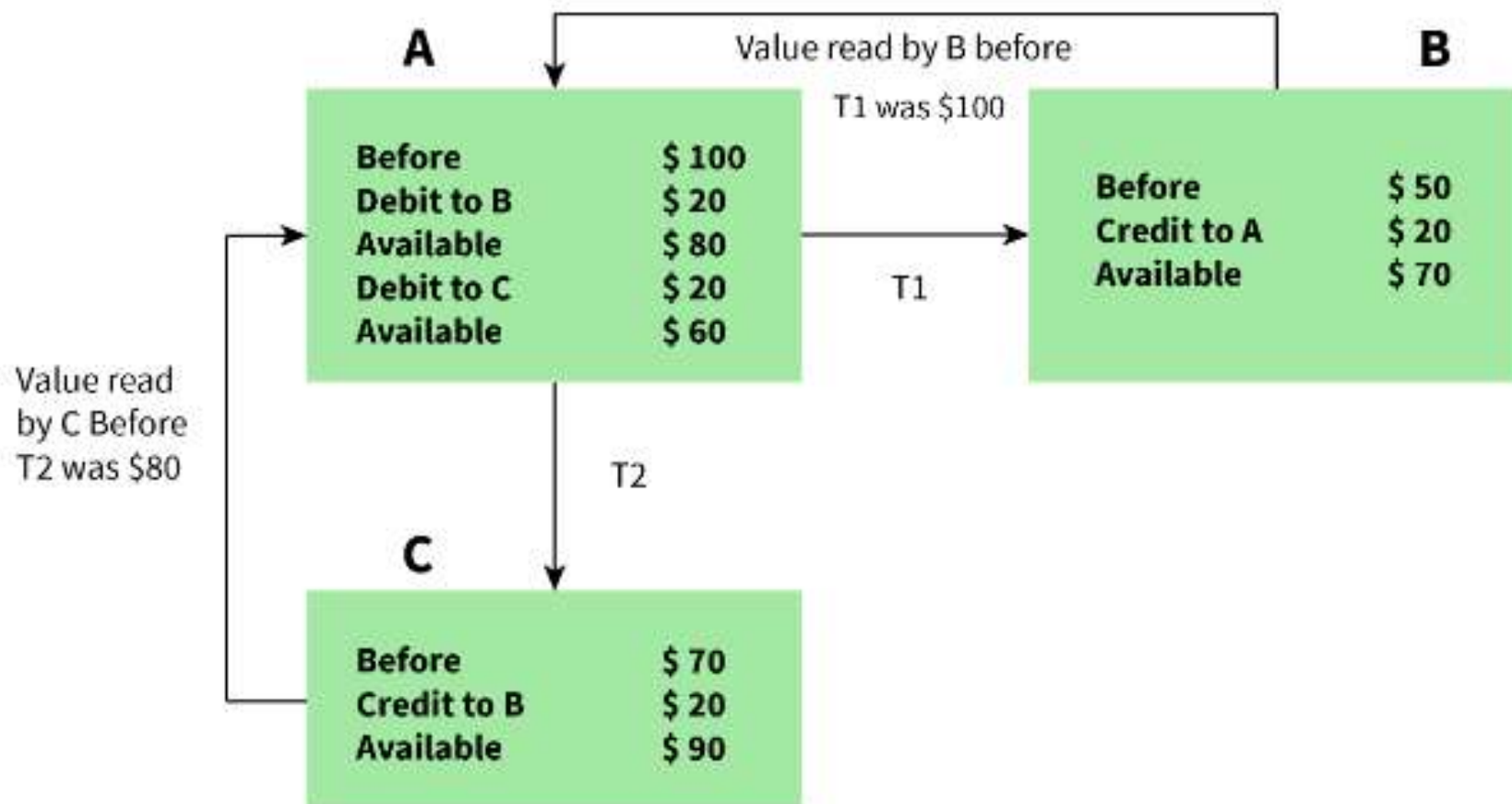
Data Consistent

Transaction Properties(ACID)

Isolation

1. Isolation is the property which guarantees that the **execution of one transaction should not interfere with the execution of another transaction.**
2. Isolation property **does not ensure the order** of transactions.
3. The operations **cannot access or see the data** in an **intermediate state** during a transaction.
4. **Isolation is needed** when there are concurrent transactions occurring at the same time.

Isolation

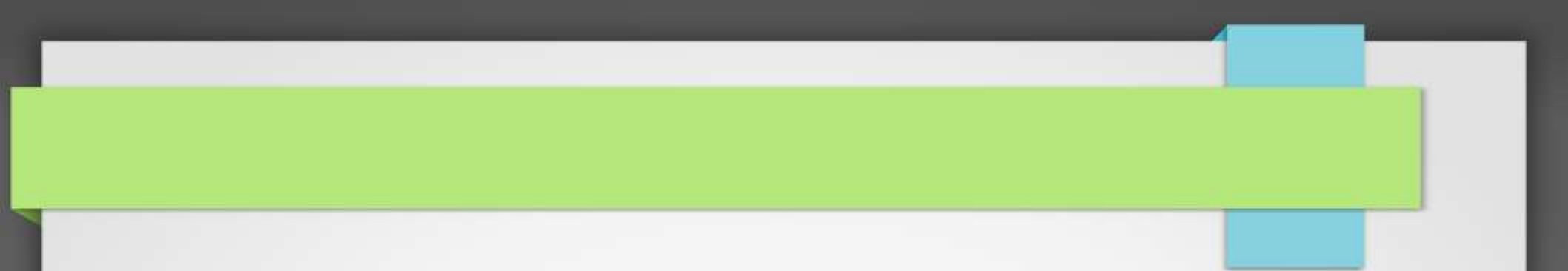


Isolation - Independent execution T1 and T2 by A

Transaction Properties(ACID)

Durability

1. Durability states that **after completion** of transaction successfully, the **changes are required for the database.**
2. Durability **holds its latest updates** even if the system fails or restarts.
3. It has the ability to **recover committed transaction updates** even if the storage media fails.



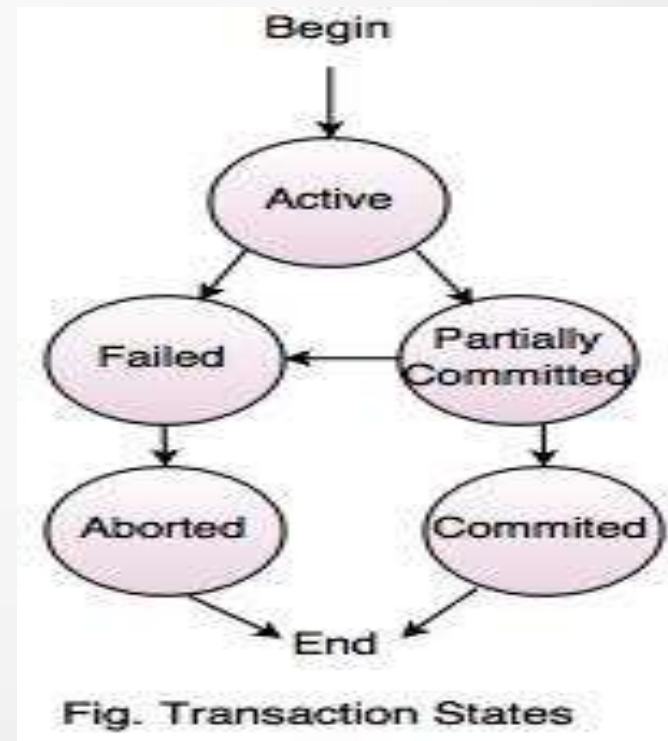
Transaction States

Transaction States

1. A transaction is a small unit of program which contains several low level tasks.
2. It is an event which occurs on the database.

It has the following states,

1. Active
2. Partially Committed
3. Failed
4. Aborted
5. Committed



Transaction States

Active – The initial state; the transaction stays in this state while it is executing

Partially committed – after the final statement has been executed.

Failed -- after the discovery that normal execution can no longer proceed.

Aborted – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction.

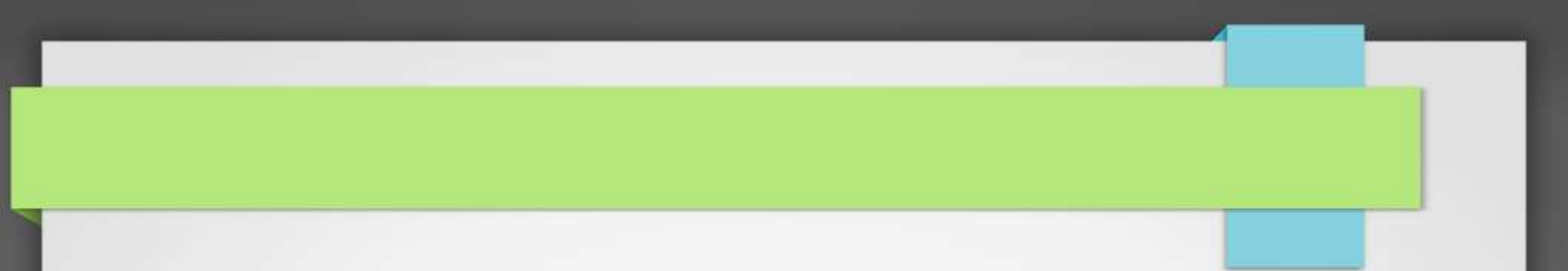
Two options after it has been aborted:

Restart the transaction

- can be done only if no internal logical error

Kill the transaction

Committed – after successful completion.



Concurrency control

WHAT IS CONCURRENCY CONTROL?

CONCURRENCY CONTROL

The **coordination of the simultaneous execution of transactions** in a multiuser database system is known as **concurrency control**.

Objective of concurrency control

The **objective** of concurrency control is to **ensure the serializability of transactions** in a multiuser database environment.

Concurrency control

Concurrency control is important because the simultaneous execution of transactions over a shared database can **create several data integrity and consistency problems.**

The three main problems are

Lost updates, (WW Conflict)

Uncommitted data, (WR conflict-Dirty Read Problems)

Inconsistent retrievals. (RW Conflict)

Phantom Read Problem

LOST UPDATES (WW CONFLICT)

The **lost update** problem occurs when **two concurrent transactions, T1 and T2, are updating the same data element** and one of the updates is lost (**overwritten by the other transaction**).

Lost Updates (WW Conflict)

For example:

Consider the below diagram where two transactions T_x and T_y , are performed on the same account **A** where the balance of account A is **\$300**.

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A - 50$	
t_3	—	READ (A)
t_4	—	$A = A + 100$
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

LOST UPDATE PROBLEM

Lost Updates (WW Conflict)

1. At time t_1 , transaction T_X reads the value of account A, i.e., \$300 (only read).
2. At time t_2 , transaction T_X deducts \$50 from account A that becomes \$250 (only deducted and not updated/write).
3. Alternately, at time t_3 , transaction T_Y reads the value of account A that will be \$300 only because T_X didn't update the value yet.
4. At time t_4 , transaction T_Y adds \$100 to account A that becomes \$400 (only added but not updated/write).
5. At time t_6 , transaction T_X writes the value of account A that will be updated as \$250 only, as T_Y didn't update the value yet.
6. Similarly, at time t_7 , transaction T_Y writes the values of account A, so it will write as done at time t_4 that will be \$400. It means the value written by T_X is lost, i.e., \$250 is lost.

Uncommitted Data(WR conflict-Dirty Read Problems)

Dirty Read Problems

The phenomenon of **uncommitted data** occurs when two transactions, **T1** and **T2**, are executed concurrently and the first transaction (**T1**) is rolled back after the second transaction (**T2**) has already accessed the **uncommitted data**—thus violating the isolation property of transactions.

Uncommitted Data(WR conflict)

For example:

Consider two transactions T_X and T_Y in the below diagram performing read/write operations on account **A where the available balance in account A is \$300:**

Time	T_X	T_Y
t_1	READ (A)	—
t_2	$A = A + 50$	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

DIRTY READ PROBLEM

Uncommitted Data(WR conflict)

1. At time t_1 , transaction T_X reads the value of account A, i.e., \$300.
2. At time t_2 , transaction T_X adds \$50 to account A that becomes \$350.
3. At time t_3 , transaction T_X writes the updated value in account A, i.e., \$350.
4. Then at time t_4 , transaction T_Y reads account A that will be read as \$350.
5. Then at time t_5 , transaction T_X rollbacks due to server problem, and the value changes back to \$300 (as initially).
6. But the value for account A remains \$350 for transaction T_Y as committed, which is the dirty read and therefore **known as the Dirty Read Problem.**

Inconsistent Retrievals(RW Conflict)

Unrepeatable Read Problem

This problem occurs when a transaction gets to read unrepeated i.e. different values of the same variable in its different read operations even when it has not updated its value.

Inconsistent Retrievals(RW Conflict)

Example

Transaction T1	Transaction T2
R (X)	
	R (X)
W (X)	
	R (X) // Unrepeated Read

T1 reads the value of X (= 10 say).

T2 reads the value of X (= 10).

T1 updates the value of X (from 10 to 15 say) in the buffer.

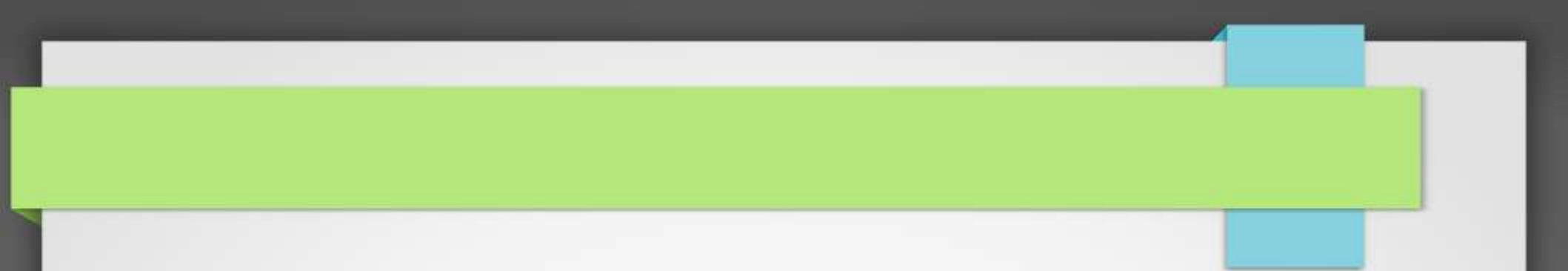
T2 again reads the value of X (but = 15).

Phantom Read Problem

This problem occurs when a transaction reads some variable from the buffer and when it reads the same variable later, it finds that the variable does not exist.

Transaction T1	Transaction T2
R (X)	
	R (X)
Delete (X)	
	Read (X)

Here,
T1 reads X.
T2 reads X.
T1 deletes X.
T2 tries reading X but does not find it.



TRANSACTIONS AND SCHEDULES

Schedules

Schedule – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed.

1. A schedule for a set of transactions must consist of all instructions of those transactions.

A transaction that **successfully completes** its execution will have a **commit** instructions as the last statement

Schedule 1

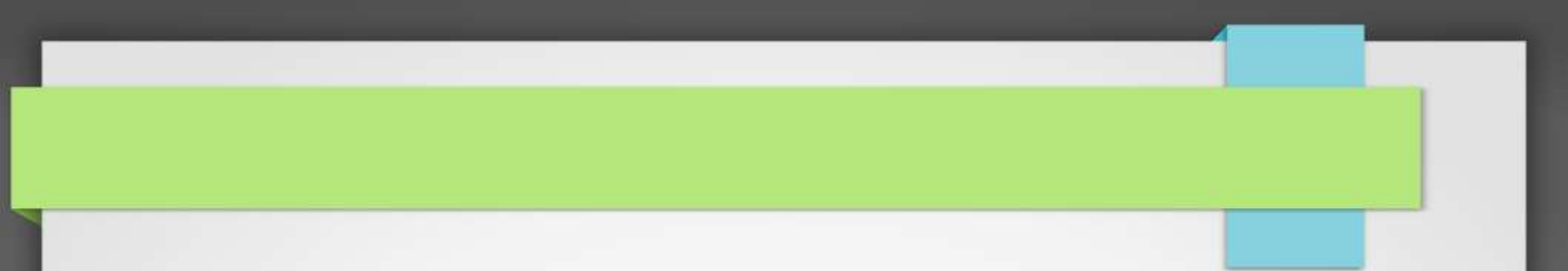
Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .

An example of a **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

A **serial** schedule in which T_2 is followed by T_1 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



Serializability

What is Serializability?

1. Serializability is related to schedules and transactions.
2. Schedule is a set of transactions.
3. Transaction is a set of instructions used to perform any logical operations in terms of databases.

What is Serializability?

Serializability of schedules ensures that a **non-serial schedule is equivalent to a serial schedule.**

It helps in maintaining the transactions to execute simultaneously without interleaving one another.

Types of Schedules in DBMS

There are two types of schedules in DBMS:

- 1. Serial schedule**
- 2. Non-serial schedule**

Types of Schedules in DBMS

Serial schedule

These are those schedules in which transactions are executed sequentially, meaning a new transaction doesn't begin until the previous one has concluded.

Example

Transaction	Operation
T1	Read(X)
T1	Write(X)
T2	Read(Y)
T2	Write(Y)

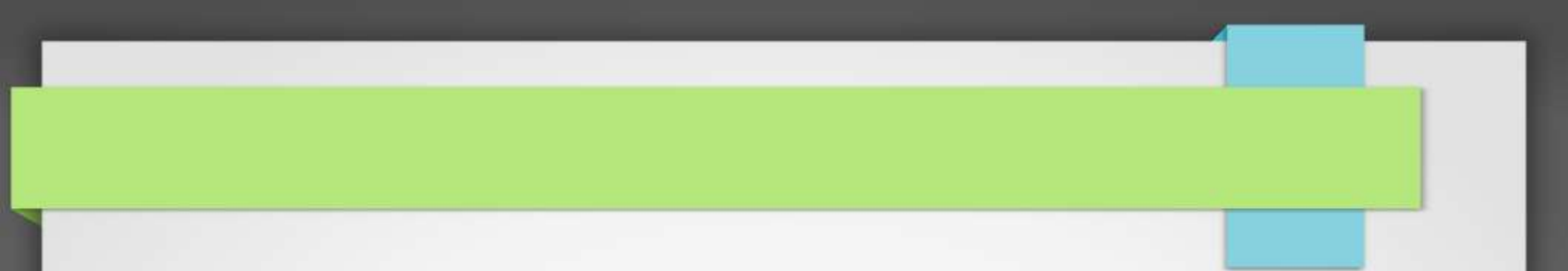
Types of Schedules in DBMS

Non-serial schedule

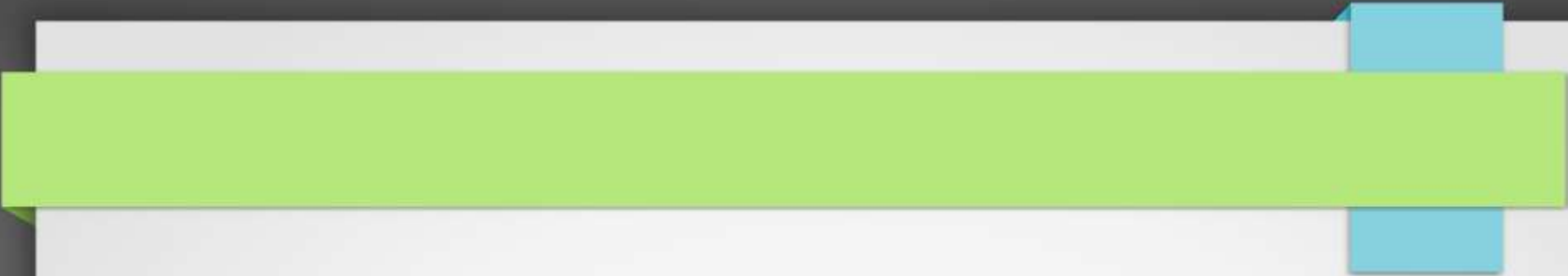
1. This scheduling method involves interleaving the operations of multiple transactions, potentially increasing concurrency challenges.
2. Transactions are executed concurrently, yet the final outcome remains consistent and identical to a serial schedule.

Example

Transaction	Operation
T1	Read(X)
T2	Read(Y)
T1	Write(X)
T2	Write(Y)



Types of serializability



There are two types of serializability –

1. View serializability

2. Conflict serializability

View serializability

A schedule is view-serializability if it is viewed equivalent to a serial schedule.

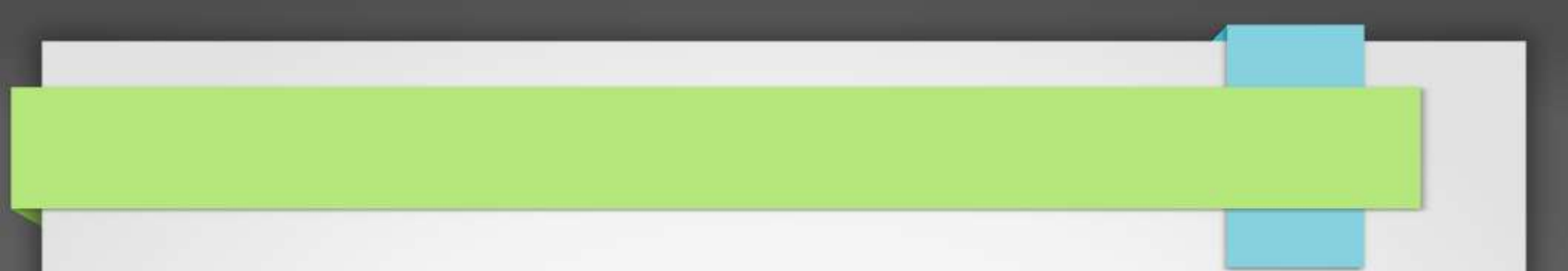
The rules it follows are as follows –

- T1 is reading the initial value of A, then T2 also reads the initial value of A.
- T1 is the reading value written by T2, then T2 also reads the value written by T1.
- T1 is writing the final value, and then T2 also has the write operation as the final value.

Conflict serializability

It orders any conflicting operations in the same way as some serial execution. A pair of operations is said to conflict if they operate on the same data item and one of them is a write operation.

- $\text{Read}_i(x) \text{ read}_j(x)$ - non conflict read-read operation
- $\text{Read}_i(x) \text{ write}_j(x)$ - conflict read-write operation.
- $\text{Write}_i(x) \text{ read}_j(x)$ - conflict write-read operation.
- $\text{Write}_i(x) \text{ write}_j(x)$ - conflict write-write operation.



Testing of Serializability

Testing of Serializability

1. To test the serializability of a schedule, we can use **Serialization Graph** or **Precedence Graph**.
2. A serialization Graph is nothing but a Directed Graph of the entire transactions of a schedule.

It can be defined as a Graph $G(V, E)$ consisting of a set of directed-edges $E = \{E_1, E_2, E_3, \dots, E_n\}$ and a set of vertices $V = \{V_1, V_2, V_3, \dots, V_n\}$.

The set of edges contains one of the two operations - **READ**, **WRITE** performed by a certain transaction.

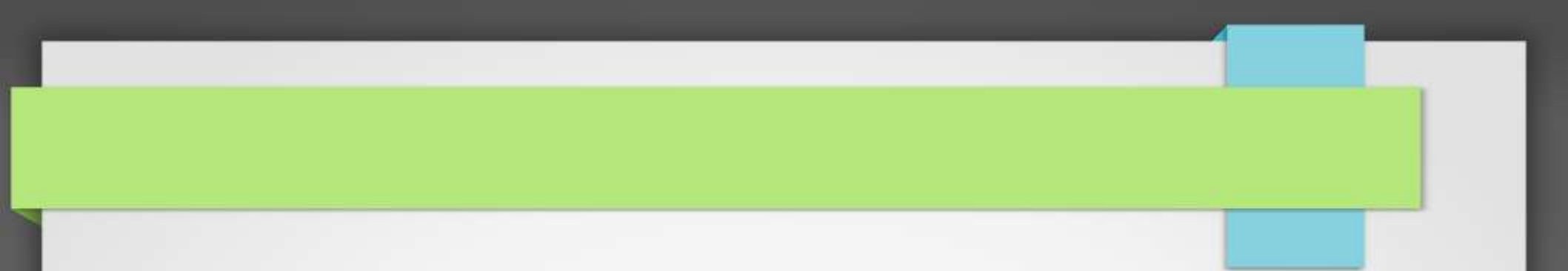
Testing of Serializability

Precedence Graph for Schedule S



$T_i \rightarrow T_j$, means

Transaction- T_i is either performing read or write before the transaction- T_j .



Recoverability of Schedule

Recoverability of Schedule

1. Sometimes a transaction may not execute completely due to a software issue, system crash or hardware failure. In that case, the failed transaction has to be rollback.
2. But some other transaction may also have used value produced by the failed transaction. So we also have to rollback those transactions.

Recoverability of Schedule

Irrecoverable schedule: The schedule will be irrecoverable if T_j reads the updated value of T_i and T_j committed before T_i commit.

Recoverable with cascading rollback: The schedule will be recoverable with cascading rollback if T_j reads the updated value of T_i . Commit of T_j is delayed till commit of T_i .

Recoverability of Schedule

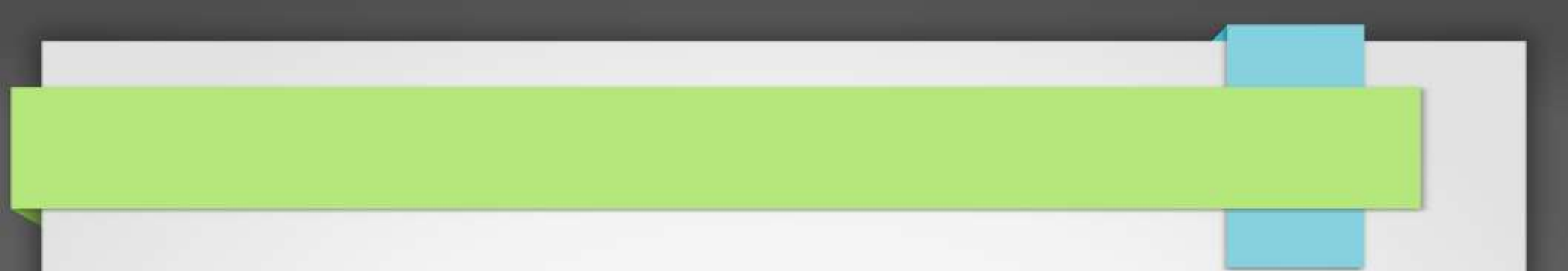
Irrecoverable schedule: The schedule will be irrecoverable if T2 reads the updated value of T1 and Tj committed before T1 commit.

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
		Read(A);	A = 6000	A = 6000
		A = A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
		Commit;		
Failure Point				
Commit;				

Recoverability of Schedule

Recoverable with cascading rollback: The schedule will be recoverable with cascading rollback if T2 reads the updated value of T1. Commit of T2 is delayed till commit of T1.

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
Commit;		Read(A);	A = 6000	A = 6000
		A = A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
		Commit;		



Failure Classification

Failure Classification

To find that where the problem has occurred, we generalize a failure into the following categories:

1. **Transaction failure**
2. **System crash**
3. **Disk failure**

Failure Classification

1. Transaction failure

The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further.

If a few transaction or process is hurt, then this is called as transaction failure.

Reasons for a transaction failure could be –

Logical errors

Syntax error

Failure Classification

2. System Crash

System failure can occur due to power failure or other hardware or software failure.

Example: Operating system error.

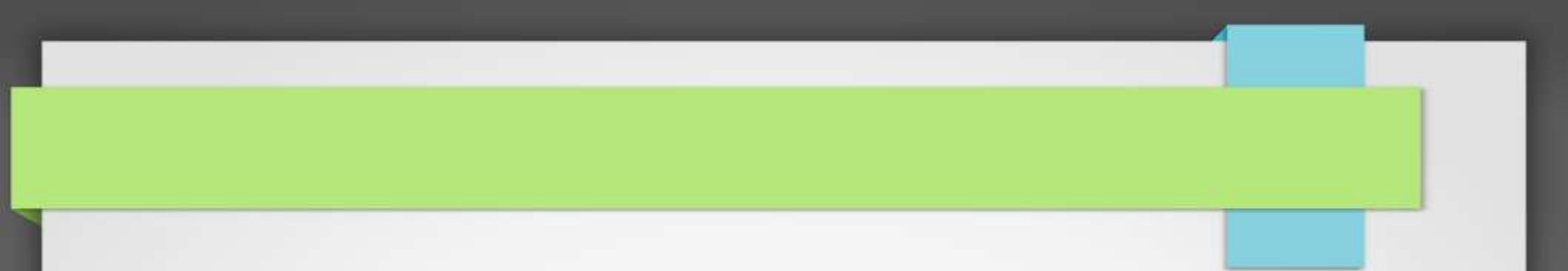
Failure Classification

3. Disk Failure

It occurs where hard-disk drives or storage drives used to fail frequently.

It was a common problem in the early days of technology evolution.

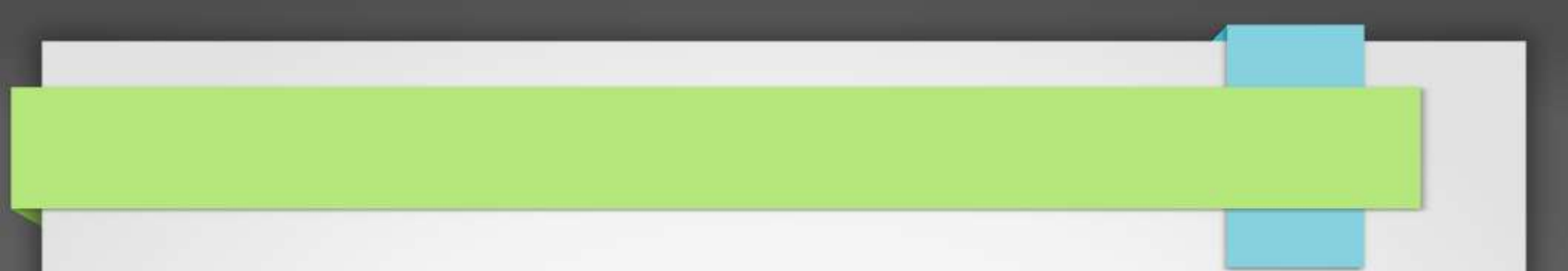
Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability to the disk or any other failure, which destroy all or part of disk storage.



Concurrency Control With Locking Methods

Concurrency Control With Locking Methods

1. A **lock** guarantees exclusive use of a data item to a current transaction.
2. In other words, transaction T2 does not have access to a data item that is currently being used by transaction T1.
3. A transaction **acquires a lock prior to data access**;
4. The lock is released (unlocked) when the **transaction is completed so that another transaction can lock** the data item for its exclusive use.

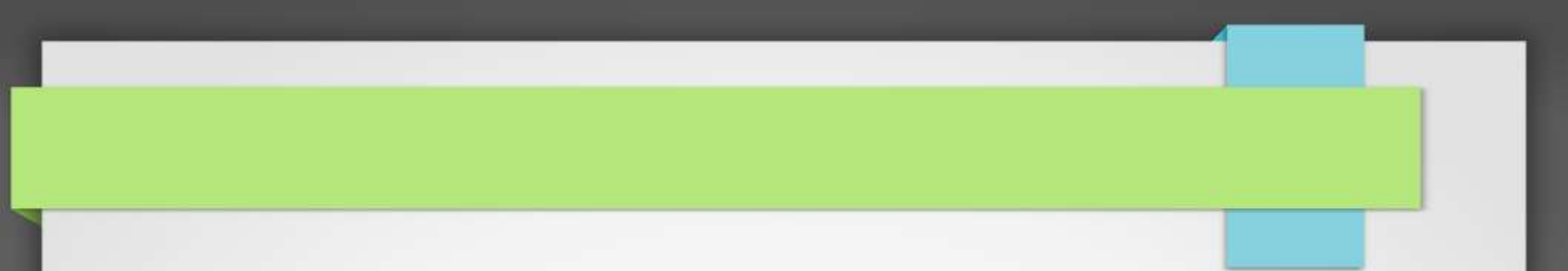


Types of Locks

Types of Locks

The DBMS may use different lock types:

1. **Binary Locks**
2. **Shared/Exclusive Locks**



Binary Locks

Types of Locks

Binary Locks

Binary Locks-**Lock with 2 States**

Locked-No other transaction can use that object

Unlocked-Any transaction can lock and use object

1. All Transactions require a Lock and Unlock Operation.
2. For Each Object Accessed(Handled by DBMS)

Binary Locks

Example of Binary Lock Table

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Lock PRODUCT	
2	T1	Read PROD_QOH	35
3	T1	$\text{PROD_QOH} = 35 + 100$	
4	T1	Write PROD_QOH	135
5	T1	Unlock PRODUCT	
6	T2	Lock PRODUCT	
7	T2	Read PROD_QOH	135
8	T2	$\text{PROD_QOH} = 135 - 30$	
9	T2	Write PROD_QOH	105
10	T2	Unlock PRODUCT	

Table 9.10

Shared and Exclusive Locks
or
Read/Write Locks

Shared and Exclusive Locks

What are Shared and Exclusive Locks

In this types of lock, system supports two kinds of lock :

Exclusive(or Write) Locks

Shared(or Read) Locks.

Shared Locks(S)

Shared Locks:

1. Shared locks are placed on resources **whenever a read operation (select) is performed.**
2. **Multiple shared locks** can be simultaneously set on a resource.

If a transaction T_i has locked the data item A in shared mode, then a request from another transaction T_j on A for :

1. **Write operation on A : Denied.** Transaction T_j has to wait until transaction T_i unlock A.
2. **Read operation on A : Allowed.**

Exclusive lock(X)

Exclusive lock:

1. Exclusive locks are placed on resources **whenever a write operation** (INSERT, UPDATE And DELETE) are performed.
2. Only **one exclusive lock** can be placed on a resource **at a time**.
3. i.e. the **first user who acquires an exclusive lock will continue to have the sole ownership** of the resource, and no other user can acquire an exclusive lock on that resource

Exclusive lock:

Exclusive Locks

If a transaction T_i has locked a data item a in exclusive mode then request from some another transaction T_j for-

Read operation on A : Denied

Write operation on A : Denied

Lock compatibility

The relationship between Shared and Exclusive Lock can be represented by the following table.

Which is known as **Lock Matrix**.

		Locks already existing	
Locks to Be granted		Shared	Exclusive
	Shared	TRUE	FALSE
	Exclusive	FALSE	FALSE

Concurrency protocols

Two Phase Locking
(2PL)

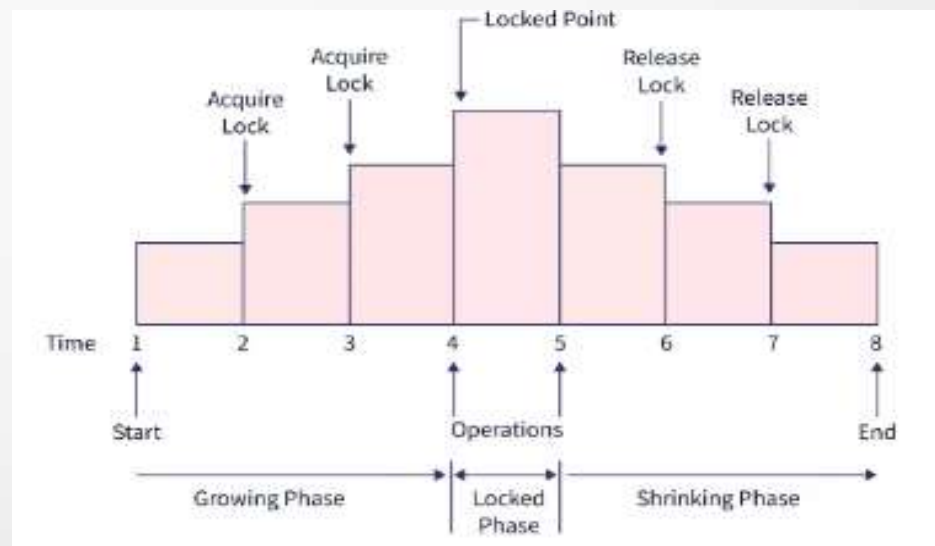
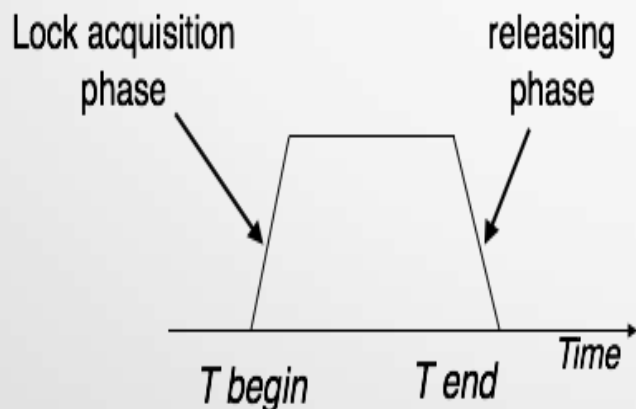
Two Phase Locking (2PL)

1. The use of locks has helped us to create **neat and clean concurrent schedule**.
2. The Two Phase Locking Protocol defines the **rules of how to acquire the locks** on a data item and how to release the locks.
3. The Two Phase Locking Protocol assumes that a transaction can only be in one of **two phases**.
 1. Growing Phase
 2. Shrinking Phase

Two Phase Locking (2PL)

In this scheme, each transaction makes lock and unlock request in 2 phases :

1. **A Growing Phase(or An Expanding Phase or First Phase)** : In this phase, new locks on the desired data item can be acquired but none can be released.
2. **A Shrinking Phase (or Second Phase)** : In this phase, existing locks can be released but no new locks can be acquired.



Two Phase Locking (2PL)

There are two different versions of the Two Phase Locking Protocol.

1. **Strict Two Phase Locking Protocol**
2. **Rigorous Two Phase Locking Protocol.**

Strict Two-Phase Locking

A schedule will be in Strict 2PL if

1. It must satisfy the basic 2-PL.
2. And each transaction should holds all **Exclusive(X) Locks** until the Transaction is Commits or abort.

T1	T2
Lock-X(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Wait
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A) ▼	
Unlock (B) ▲	Lock-S(A) // Granted
	READ (A)
	WRITE (A)
	Commit (A)
	Unlock (A)

Strict 2-PL

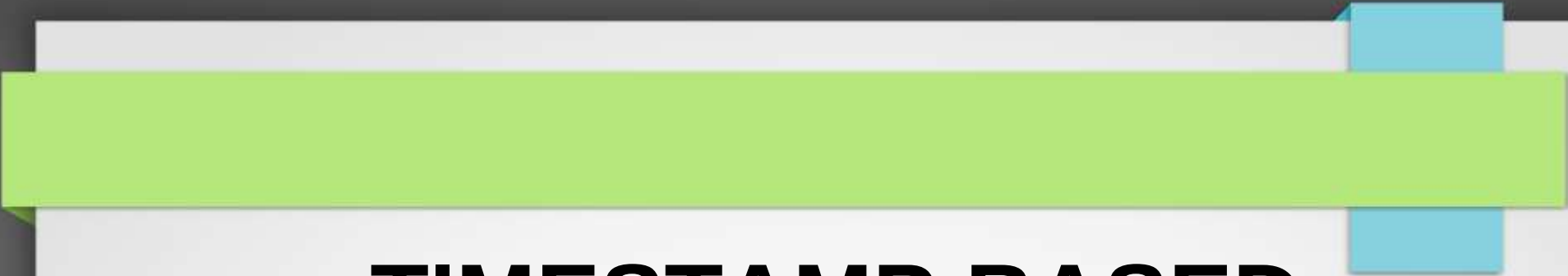
Rigorous Two-Phase Locking

A schedule will be in Rigorous 2PL if

1. It must satisfy the basic 2-PL.
2. And each transaction should holds **all Exclusive(X) Locks** as well as **Shared(S) Locks** until the Transaction is Commits or abort.

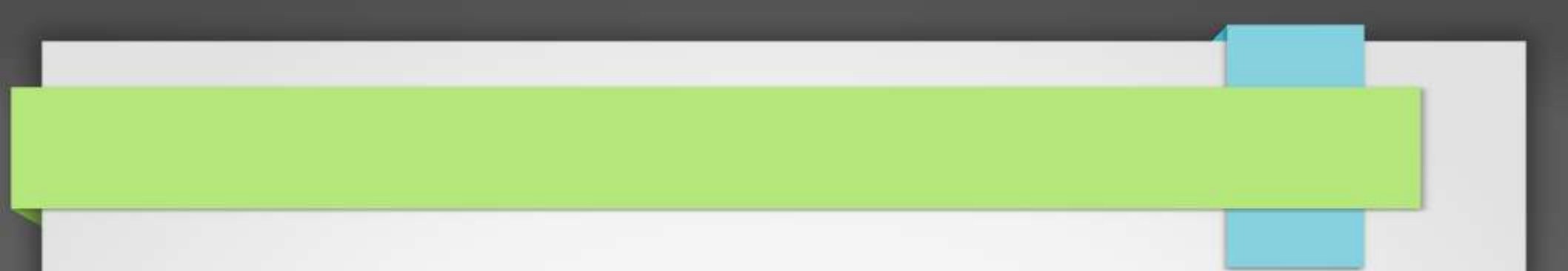
T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Wait
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A) ▼	▼
Unlock (B)	Lock-S(A) // Granted
	READ (A)
	WRITE (A)
	Commit (A)
	Unlock (A)

Rigorous 2-PL



TIMESTAMP BASED PROTOCOL :(Timestamp based concurrency control techniques)

Refer Document

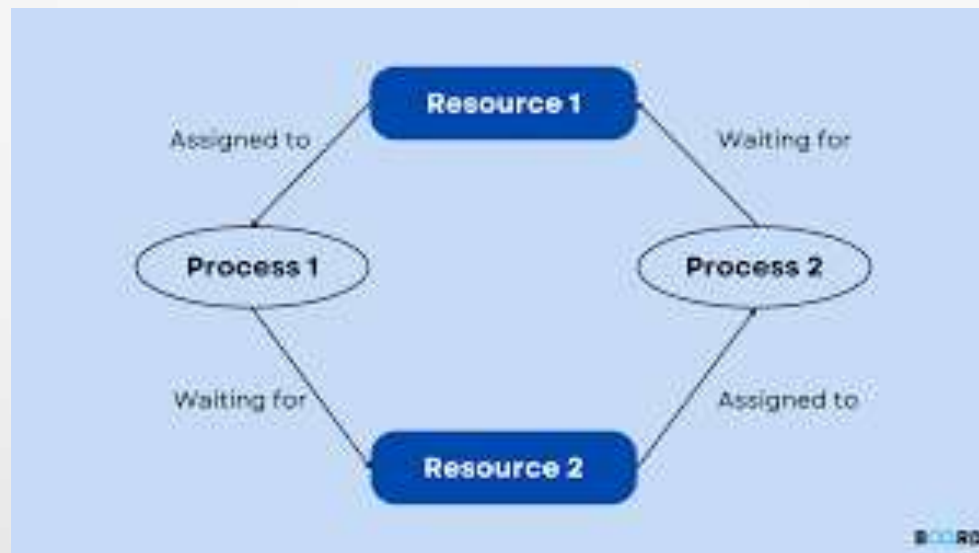


Deadlocks

What are Deadlocks?

Deadlock is a state of a database system having two or more transactions, when **each transaction is waiting for a data item** that is being locked by some other transaction.

1. A deadlock can be indicated by a cycle in the **wait-for-graph**.

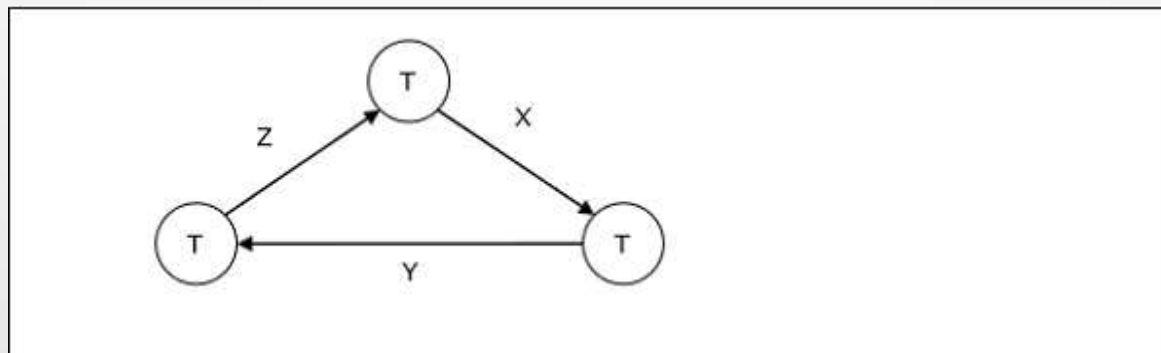


For example,

For example, in the following wait-for-graph, transaction T1 is waiting for data item X which is locked by T3.

T3 is waiting for Y which is locked by T2 and T2 is waiting for Z which is locked by T1.

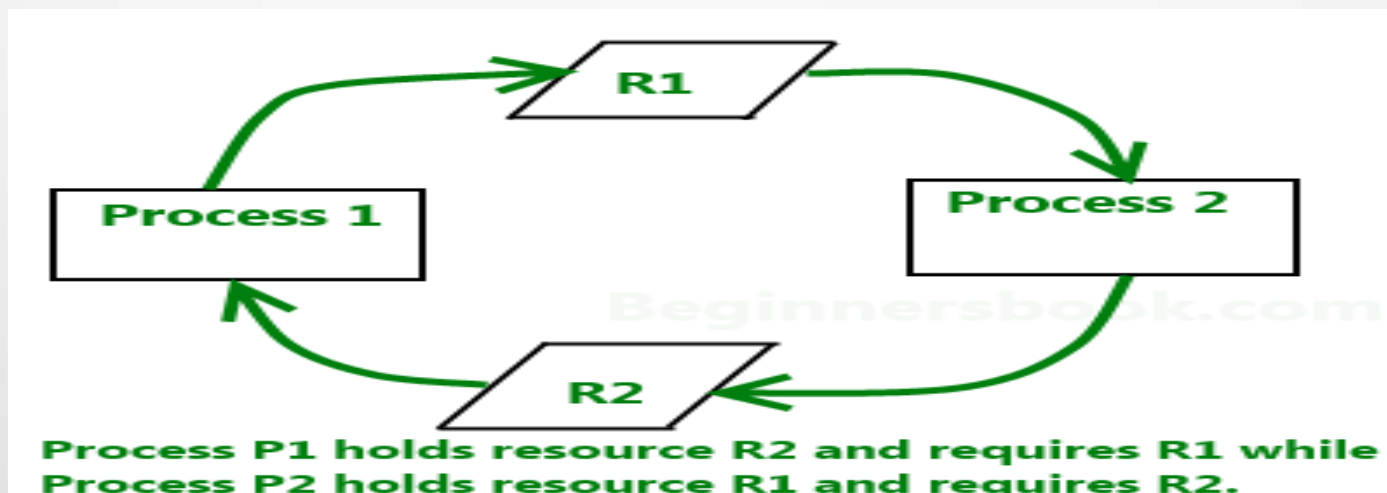
Hence, a waiting cycle is formed, and none of the transactions can proceed executing.



Coffman conditions

Coffman stated four conditions for a deadlock occurrence. A deadlock may occur if all the following conditions holds true.

1. Mutual exclusion condition:
2. Hold and wait condition
3. No preemption condition
4. Circular wait condition:

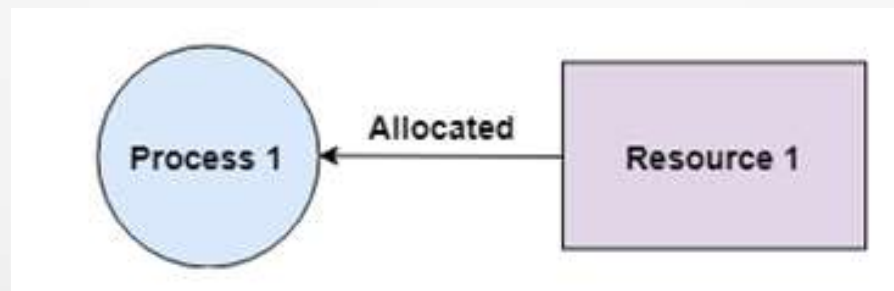


Necessary conditions for Deadlocks

Mutual exclusion

A resource can only be shared in **mutually exclusive manner**. It implies, if two process cannot use the same resource at the same time.

In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.

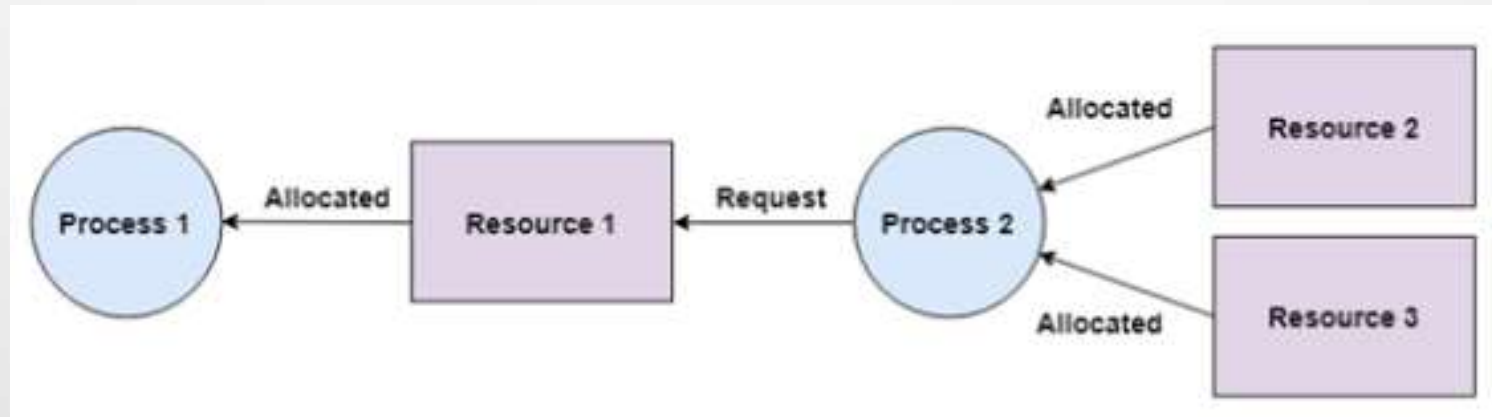


Necessary conditions for Deadlocks

Hold and wait

A process can hold multiple resources and still request more resources from other processes which are holding them.

In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.

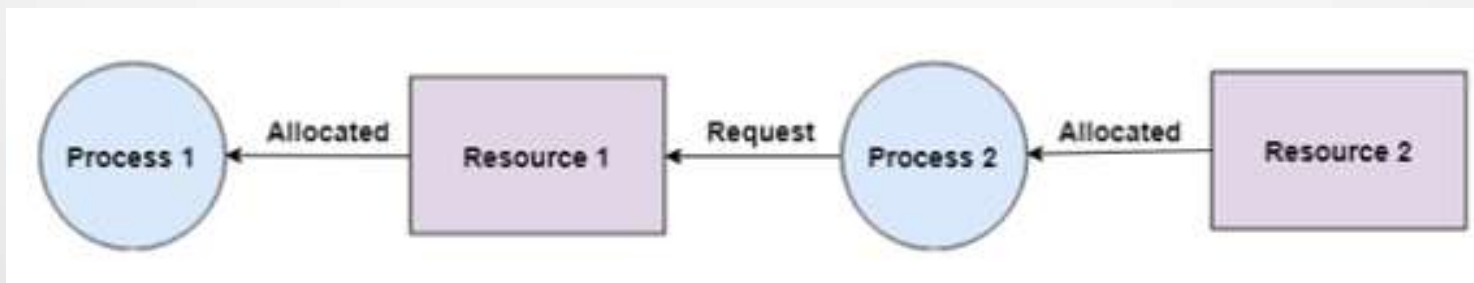


Necessary conditions for Deadlocks

No preemption

A resource **cannot be preempted** from a process by force. A process can **only release a resource voluntarily**.

In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.

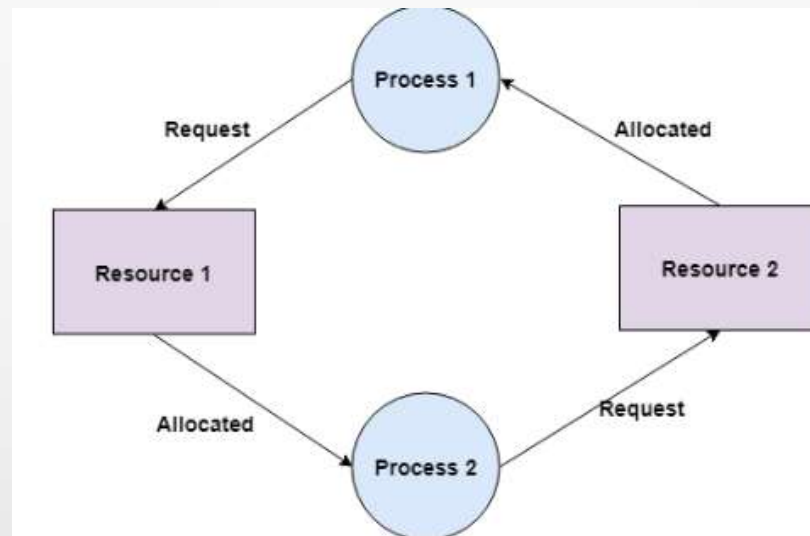


Necessary conditions for Deadlocks

Circular wait.

All the processes must be **waiting for the resources in a cyclic manner** so that the last process is waiting for the resource which is being held by the first process.

For example: Process 1 is allocated Resource 2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.



Deadlock Handling

There are three classical approaches for deadlock handling, namely –

1. **Deadlock prevention.**
2. **Deadlock avoidance.**
3. **Deadlock detection and removal.**

All of the three approaches can be incorporated in both a centralized and a distributed database system.

•Deadlock detection.

- The DBMS periodically tests the database for deadlocks.
- If a deadlock is found, one of the transactions (the “victim”) is aborted (rolled back and restarted) and the other transaction continues.

There are two algorithms for deadlock avoidance.

Wait/Die

Wound/Wait

Wait/Die & Wound/Wait

Let us assume that there are two transactions, T1 and T2, where T1 tries to lock a data item which is already locked by T2. The algorithms are as follows –

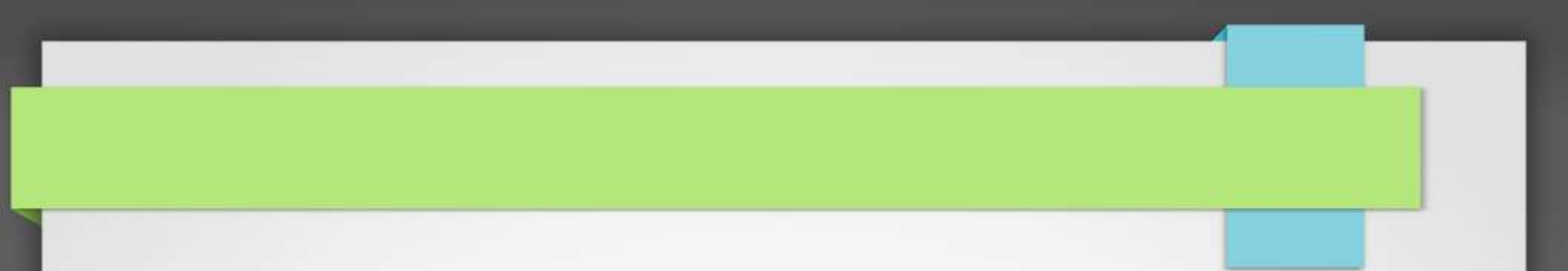
Wait-Die – If T1 is older than T2, T1 is allowed to wait. Otherwise, if T1 is younger than T2, T1 is aborted and later restarted.

Wound-Wait – If T1 is older than T2, T2 is aborted and later restarted. Otherwise, if T1 is younger than T2, T1 is allowed to wait.

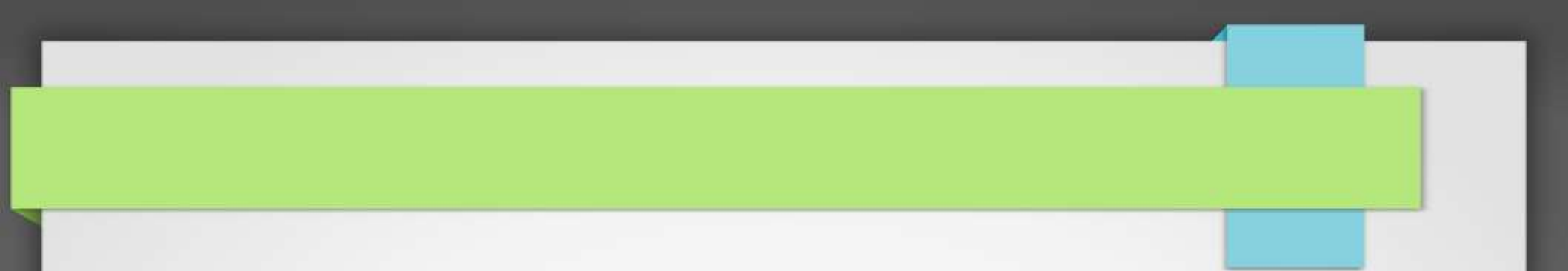
Wait/Die & Wound/Wait

Here is the table representation of resource allocation for each algorithm. Both of these algorithms take process age into consideration while determining the best possible way of resource allocation for deadlock avoidance.

	Wait/Die	Wound/Wait
Older process needs a resource held by younger process	Older process waits	Younger process dies
Younger process needs a resource held by older process	Younger process dies	Younger process waits



Introduction to Indexing Techniques



Hashing OR Hash based indexing

Hashing

1. Hash File organization method is the one where **data is stored at the data blocks** whose address is generated by using hash function.
2. The memory location where **these records are stored** is called as data block or data bucket.
3. This data bucket is capable of **storing one or more records**.
4. The **hash function** can use any of the column value to generate the address.

$$h(k)=k \bmod m$$

Hash Organization

Bucket – A hash file stores data in bucket format.

Bucket is considered a unit of storage. A bucket typically stores one complete disk block, which in turn can store one or more records.

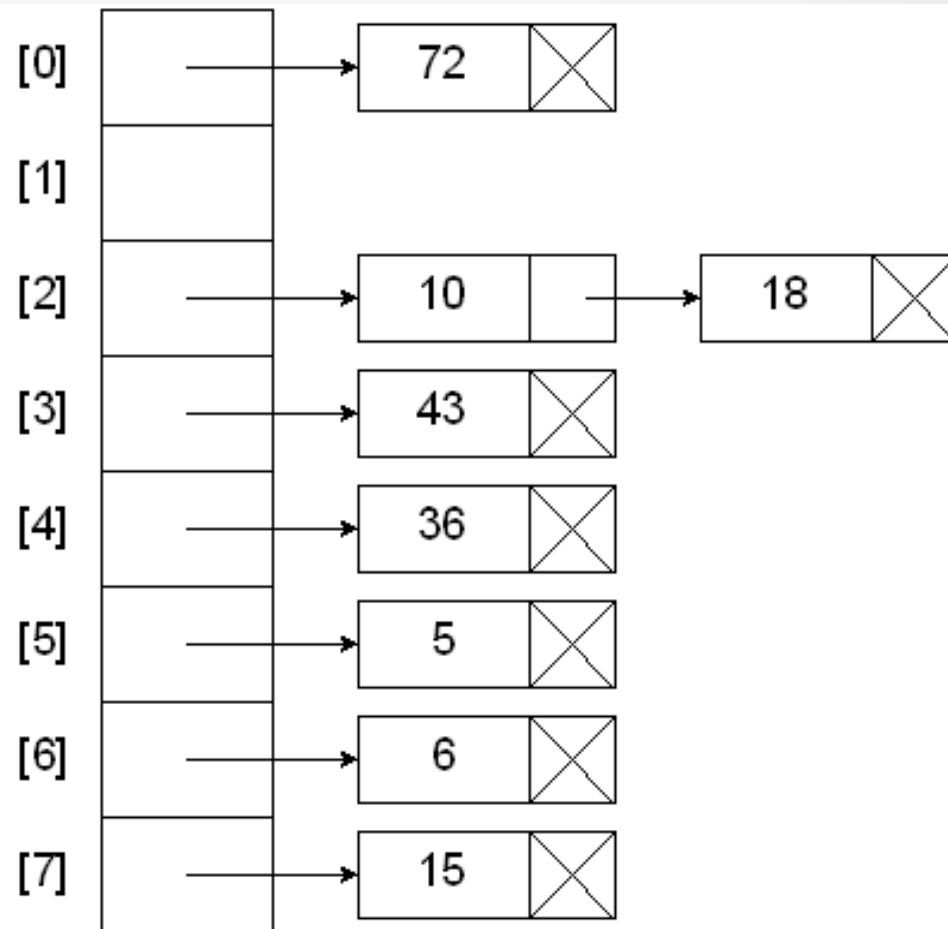
Hash Function

– A hash function, **h** , is a mapping function that maps all the set of search-keys **K** to the address where actual records are placed.

hash function

Hash key = key % table size

4	=	36	%	8
2	=	18	%	8
0	=	72	%	8
3	=	43	%	8
6	=	6	%	8
2	=	10	%	8
5	=	5	%	8
7	=	15	%	8



Types Of Hash File Organizations

There are two types of hash file organizations –

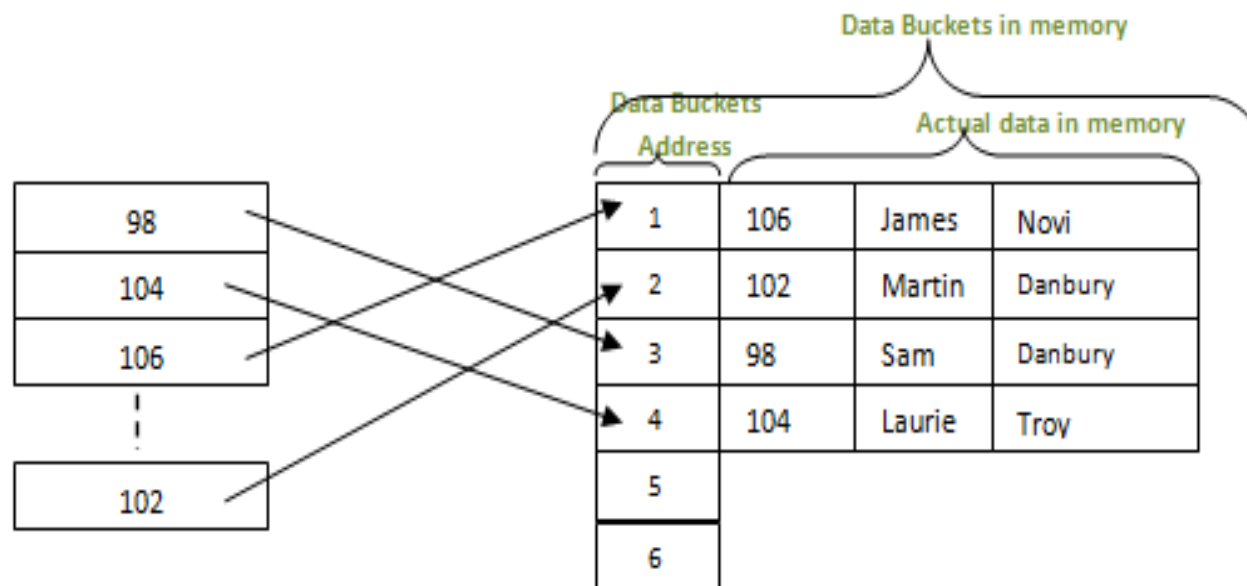
- ✓ Static Hashing
- ✓ Dynamic Hashing.

Static Hashing

1. In this method of hashing, the **resultant data bucket address will be always same.**
2. That means, if we want to generate address for **EMP_ID = 103** using **mod (5)** hash function, it always result in the same bucket address 3.
3. There **will not be any changes** to the bucket address here.
4. Hence number of data buckets in the memory for this **static hashing remains constant throughout.**

Static Hashing

In our example, we will have five data buckets in the memory used to store the data.



Static Hashing

Operations:

Insertion – When a record is required to be entered using static hash, the hash function **h** computes the bucket address for search key **K**, where the record will be stored.

$$\text{Bucket address} = h(K)$$

Search – When a record needs to be retrieved, the same hash function can be used to retrieve the address of the bucket where the data is stored.

Delete – This is simply a search followed by a deletion operation.

Static Hashing drawback in this method.

Bucket Overflow:

This is one of the critical situations/ drawback in this method.

Suppose we have to insert some records into the file. But the data bucket address generated by the hash function is full or the data already exists in that address. How do we insert the data?

This situation in the static hashing is called **bucket overflow**

Static Hashing

Bucket Overflow

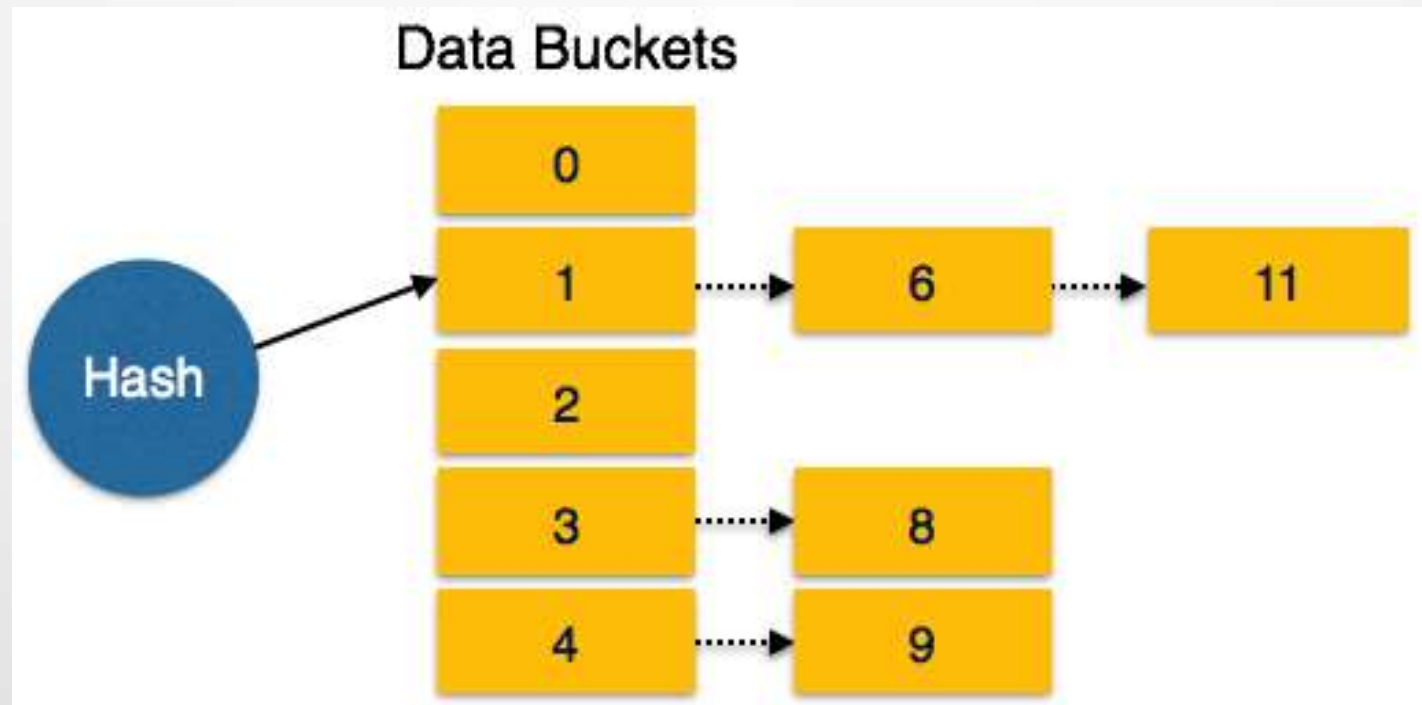
✓ The condition of bucket-overflow is known as **collision**.

This is a fatal state for any static hash function. In this case, we use.

1. Overflow Chaining(Closed Hashing)
2. Linear Probing(Open Hashing)

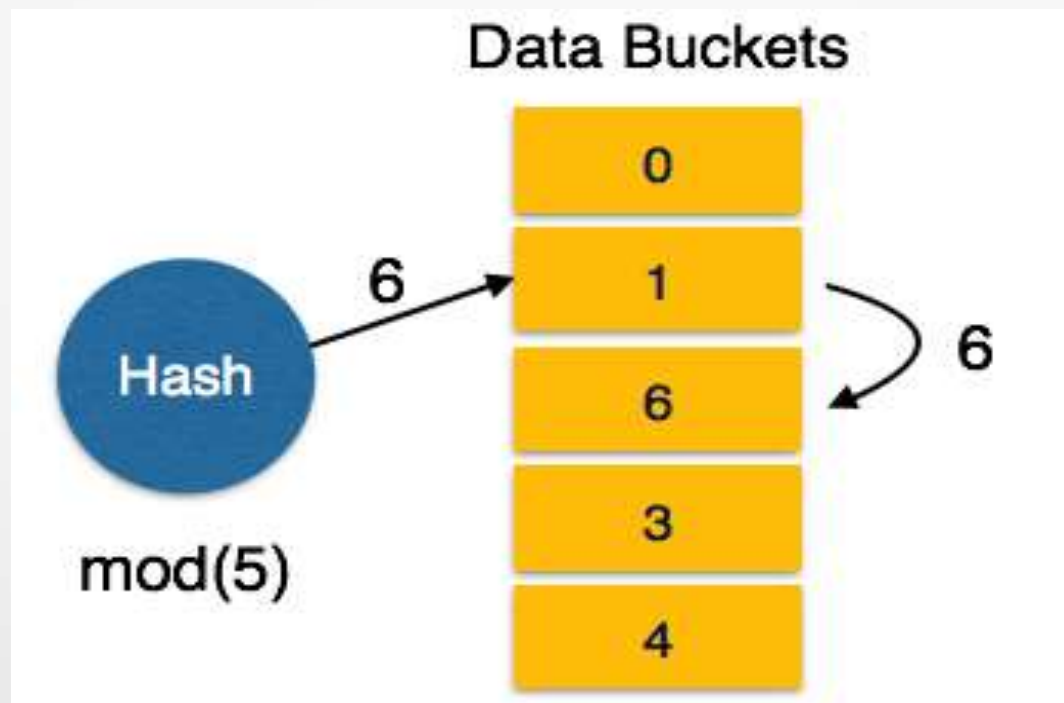
Static Hashing

Overflow Chaining – When buckets are full, a new bucket is allocated for the same hash result and is linked after the previous one. This mechanism is called **Closed Hashing**.



Static Hashing

Linear Probing – When a hash function generates an address at which data is already stored, the next free bucket is allocated to it. This mechanism is called **Open Hashing**.



Dynamic Hashing

1. This hashing method is used to overcome the problems of static hashing – **bucket overflow**.
2. In this method of hashing, **data buckets grows or shrinks** as the records increases or decreases.
3. This method of hashing is also known as **extendable hashing method**.
4. Dynamic hashing provides a mechanism in which **data buckets are added and removed dynamically** and on-demand.

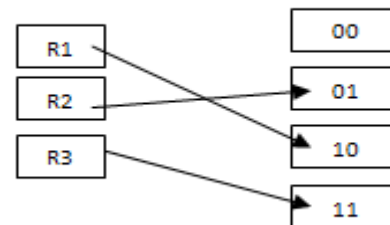
Example to understand this method.

What will happen to R3 here?

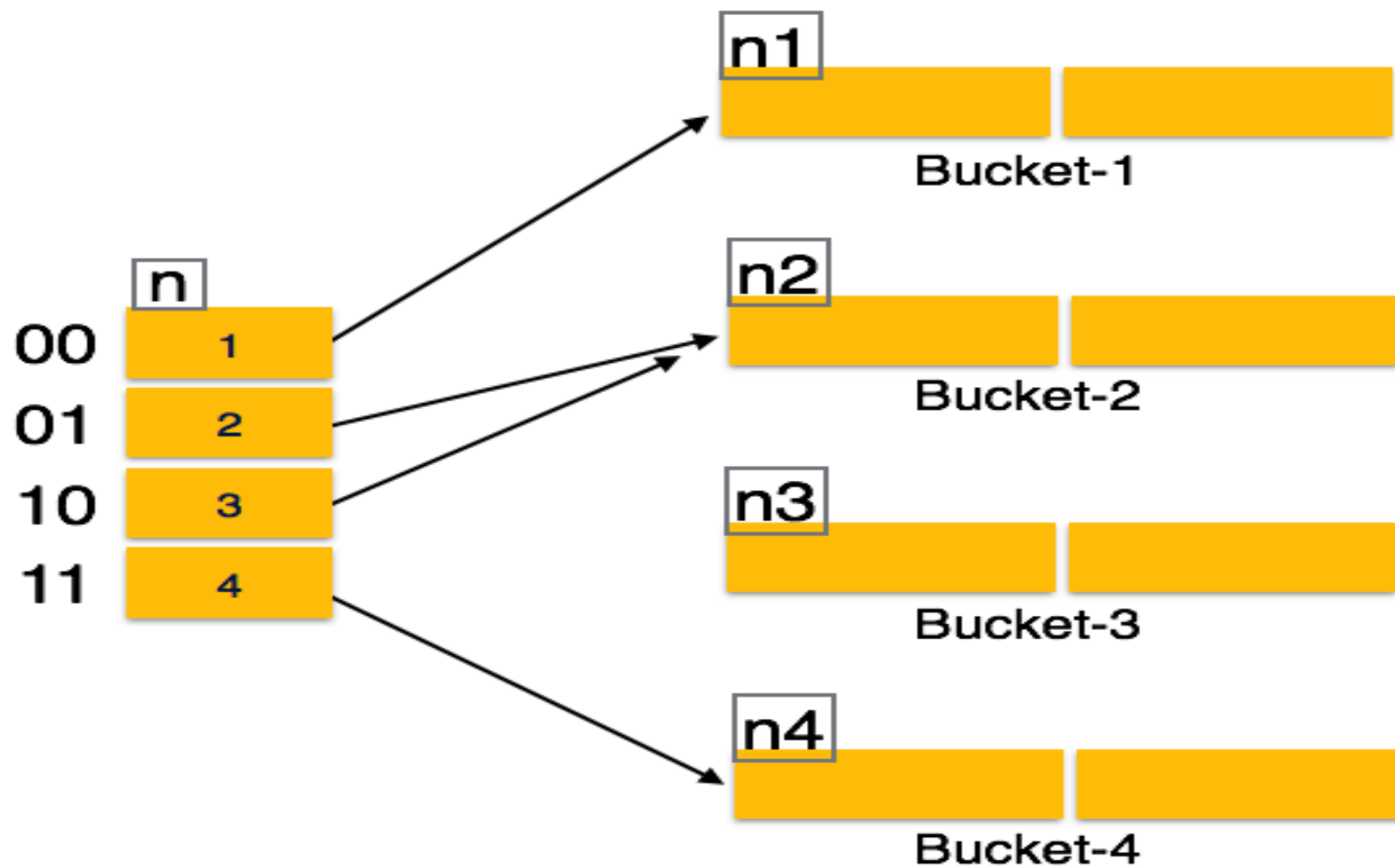
1. There is **no bucket space** for R3.
2. The bucket has to **grow dynamically** to accommodate R3.
3. So it **changes the address have 2 bits rather than 1 bit**, and then it updates the existing data to have 2 bit address.

Then it tries to accommodate R3.

$h(R1) \rightarrow 100100$
 $h(R2) \rightarrow 010110$
 $h(R3) \rightarrow 110110$

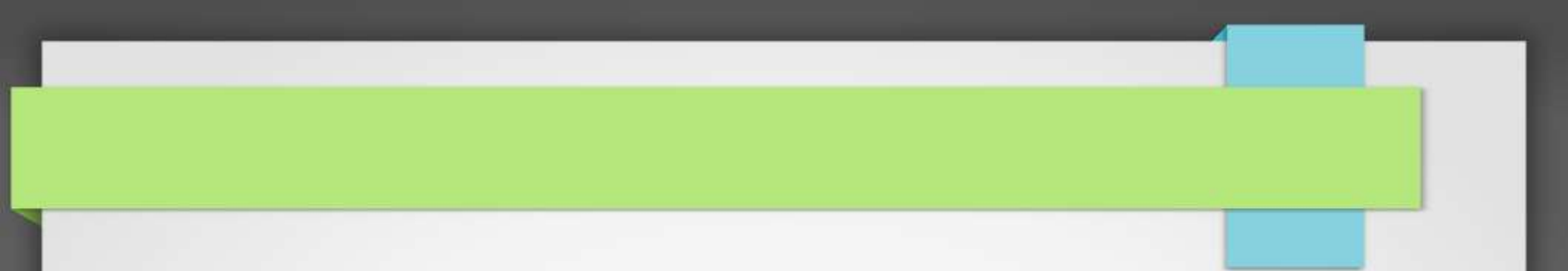


Example to understand this method.



Advantages of Dynamic hashing

1. Performance does not come down as the data grows in the system. It simply increases the memory size to accommodate the data.
2. Since it grows and shrinks with the data, memory is well utilized. There will not be any unused memory lying.
3. Good for dynamic databases where data grows and shrinks frequently.



For B+ Trees
Refer Document